Microprocessors : 8086/8088

8086

- 20 bit address bus
  16 bit data bus

8088

- 20 bit address bus
  8 bit data bus (allows usage of cheaper & fewer supporting logic chips)

Both have same instruction set

Q. Difference between 8086 & 8088

Q. Similarities " " " "

Q. Difference between 8085 and 8086/8088

Q. Similarities " " " "

Q. Difference between flag registers of 8085 and 8086

Organization

Execution Unit (EU)

- ALU
- Registers (AX, BX, CX, DX, SI, DI, BP, SP) [store data] + flag [reflect results of computation]

Bus Interface Unit (BIU)

- facilitates communication bet^{n} I/O & EU
- transmit address, data, control signal
- Registers (CS, DS, ES, SS, IP) [hold addresses of locations]
- IP [contain address of next inst. to be executed by EU]

8086 Registers and Memory

Number of Registers: 14, each of that16-bit registers

Memory Size: 1M Bytes

Registers of the 8086/80286 by Category

Category	Bits	Register Names
General	16	AX,BX,CX,DX
	8	AH,AL,BH,BL,CH,CL,DH,DL
Pointer	16	SP (Stack Pointer), Base Pointer (BP)
Index	16	SI (Source Index), DI (Destination Index)
Segment	16	CS(Code Segment) DS (Data Segment) SS (Stack Segment) ES (Extra Segment)
Instruction	16	IP (Instruction Pointer)
Flag	16	FR (Flag Register)

4 (hold data)

9 (hold addrers)

(status)

General Data Registers

BH, BL

AH, AL

- These are 16-bit registers and can also be used as two 8 bit registers: ***low and high bytes*** can be accessed separately
- **AX (Accumulator)**
  - Most efficient register for arithmetic, logic operations and data transfer: the use of AX generates the shortest machine code.
  - In multiplication and division operations, one of the numbers involved must be in AL or AX
- **BX (Base)**
  - The base address register (offset)
- **CX (Counter)**
  - Counter for looping operations: loop counter, and in the shift and rotate bits
- **DX (Data)**
  - Used in multiply and divide, also used in I/O operations

Memory Segment & Offset

- 20 bit address (physical) to memory locations
- But registers are 16 bits, can address 64 KB, memory – 1 Mbyte

Solution – Memory is segmented

4 – 64 KB segments positioned within 1 MB address space

Identified by – segment no. (held by segment registers) → starting pos.

Location specified by – offset (num of bytes from beginning of segment)

(logical address)

Example –

A4FB : 4872 h

↓ segment ↓ offset

Segmented Memory Address –

Physical Address = Segment Num × 10_h + offset

(multiply by 16)

*

A4FB0 h
+ 4872 h
A9822 h (20 bits physical address)

Segment 0 →

0000 : 0000 → 00000 h

to 0000 : FFFF → 0FFFF h

Segment 1 →

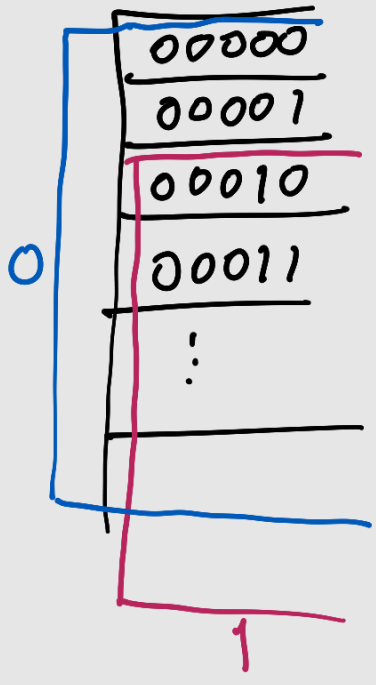
0001 : 0000 → 00010 h

to 0001 : FFFF → 1000F h

```
  00010
+ FFFF
------
 1000F
```

[to find end]

Here, overlap happens between segment 0 & 1



Segment Registers

- Four Segment Registers in the BIU are used to hold the upper 16-bits of the starting addresses of four memory segments, namely
  - **Code segment CS**: holds segment address of the code segment.
  - **Data Segment DS:** holds segment address of the data segment.
  - **Extra Segment ES:** extra segment : holds alternate segment address of the data segment.
  - **Stack Segment SS:** holds segment address of the stack segment and used when sub-program executes.
- Codes , data , and stack are loaded into different memory segments (registers).

- IP is offset relative to CS

CS = B3FFh IP = 12ABh

Physical address of next instruction

```
  B3FF0 h
+ 12AB h
---------
 B529B h.
```

- SP is offset to SS

always points to word (byte at even address)

SS = 5D27h SP = FFFEh (empty stack has this)

Phy address of top of stack data

5D270 + FFFE = 6D26E

- BP also access stack stuff → can be used in other segments too tho.

  Mainly for parameters

  offset to SS.

- SI (source index) → string operations src

  used with DS/ES

  offset to DS

- DI (Dest. index) → string ops.

  ES/DS

  offset to ES

Example, DS = E000h SI (EA) = 437Ah

↓

effective addr.

Phy addr. of data = DS + SI

Flag Register

- Carry flag CF → for unsigned overflow
- Parity flag PF
- Auxiliary flag AF → for unsigned overflow for low nibble
- Zero flag ZF
- Sign flag SF
- Trap flag → for on chip single step debugging
- Interrupt enable flag IF → CPU reacts to interrupts when set to 1
- Direction flag → used by some instructions to process data chains

  0 → forward processing
  1 → backward "
- Overflow flag → when signed overflow OF

Rule for Overflow Flag

```
Overflow Flag
-------------

The rules for turning on the overflow flag in binary/integer math are two:

1. If the sum of two numbers with the sign bits off yields a result number
   with the sign bit on, the "overflow" flag is turned on.

   0100 + 0100 = 1000 (overflow flag is turned on)

2. If the sum of two numbers with the sign bits on yields a result number
   with the sign bit off, the "overflow" flag is turned on.

   1000 + 1000 = 0000 (overflow flag is turned on)

Otherwise, the overflow flag is turned off.
 * 0100 + 0001 = 0101 (overflow flag is turned off)
 * 0110 + 1001 = 1111 (overflow flag is turned off)
 * 1000 + 0001 = 1001 (overflow flag is turned off)
 * 1100 + 1100 = 1000 (overflow flag is turned off)

Note that you only need to look at the sign bits (leftmost) of the three
numbers to decide if the overflow flag is turned on or off.
```

- For example, if register AL = 7Fh and the instruction ADD AL,1 is executed then the following happen
  - AL = 80h
  - CF = 0; there is no carry out of bit 7
  - PF = 0; 80h has an odd number of ones
  - AF = 1; there is a carry out of bit 3 into bit 4
  - ZF = 0; the result is not zero
  - SF = 1; bit seven is one
  - OF = 1; the sign bit has changed
- Can be used to transfer program control to a new memory location

```
ADD AL,1
JNZ 0100h
```

AX = 00FFH, ADD AX, 0001H. Find flags.

00FF + 0001 = 0100

Flags:

- CF = 0
- AF = 1
- ZF = 0
- SF = 0
- PF = 0
- OF = 0

FF
+01
1←00

AX = 7FFFH, ADD AX, 0001H. Find flags.

7FFF + 0001 = 8000 always check binary ✓

Flags:

1000 0000 0000 0000
↓MSB 1
SF = 1

- CF = 0
- ZF = 0
- SF = 1
- PF = 0
- AF = 1
- OF = 1

+1
0111
0000
1000 ✓

AX = FFFFH, ADD AX, 0001H. Find flags.

FFFF + 0001 = 0000

Flags:

- CF = 1
- ZF = 1
- SF = 0
- PF = 1 (even parity)
- AF = 1
- OF = 0

+1
1111
0000
0...